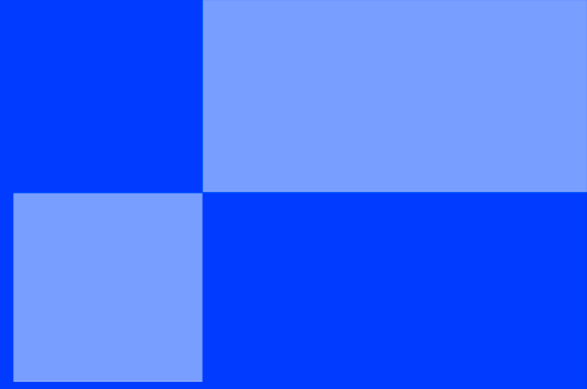




CRASH & LEARN

< MANDATORY SUBTITLE HERE />

CRASH & LEARN

Crash&Learn Grand Prix — Usage Instructions

Quick Start

```
# Build the Docker services  
docker compose build  
  
# Smoke test (runs demo.py)  
docker compose run --rm sim
```

Tests & Demo

```
# Run the built-in demo (uses submission/agent.py if present, fallback controller otherwise)  
docker compose run --rm sim
```

```
# Run contract unit tests  
docker compose run --rm sim python test_contract.py
```

```
# Run integration physics tests  
docker compose run --rm sim python test_integration.py
```

Recording & Visualization (Record + Replay)

The simulation uses a **record-then-replay** workflow: `sim_recorder.py` simulates the race at maximum speed and exports an `.npz` file, while `viz_replay.py` renders the race with interactive controls.

1. Record an episode

```
# Record a solo run (default random driver)
docker compose run --rm recorder

# Record with your trained submission
docker compose run --rm recorder python sim_recorder.py --steps 1000 --submissions submission/

# Multi-car recording (up to 4 cars)
docker compose run --rm recorder python sim_recorder.py --steps 1000 --num-cars 4 --out /app/episode_4cars.npz

# Multi-team tournament test (from a folder containing team subfolders)
docker compose run --rm recorder python sim_recorder.py --num-cars 4 --submissions /app/my_teams_folder --laps 2
```

2. Replay recorded episodes (requires display)

```
# Authorize Docker container to connect to your X server
xhost +local:docker

# Interactive replay of the latest episode
docker compose run --rm -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix replay

# Replay a specific episode with camera following car 0
docker compose run --rm -e DISPLAY=$DISPLAY -v /tmp/.X11-unix:/tmp/.X11-unix replay python viz_replay.py --npz /app/episode_4cars.npz --follow
```

Replay controls

	Key / Action	Description
Space	Play / Pause	Play / pause in real time
← / →		Step backward / forward 1 frame
Shift+← / Shift+→		Step backward / forward 10 frames
Slider		Jump to any step
Mouse Wheel		Zoom in / out centered on mouse cursor
Click + Drag		Pan across the circuit

	<u>Key / Action</u>	<u>Description</u>
	Double-Click / R	Reset camera and zoom

Training Your Agent

You create your own training pipeline (`train.py`, `env.py`, etc.). Dependencies (`torch`, `gymnasium`, `stable-baselines3`, `tyro`, `numpy`, `scipy`) are pre-installed in the Docker image.

```
# Run your training script
docker compose run --rm sim python train.py

# Launch TensorBoard in background (open http://localhost:6006)
docker compose up -d tensorboard
```

Submission structure

Local testing, a single agent To run `demo.py` or `sim_recorder.py` against your own agent, a flat `submission/` folder is enough: the loader treats it as a single team and replicates it across every car in self-play.

```
submission/
├── model.onnx
└── agent.py
```

Project delivery The delivery format is a different one: one subfolder per driver, plus a `champion` folder from Part 3 onwards. It is described in [the project subject](#).

`agent.py` must declare an `Agent` class exposing `predict(obs: dict, info: dict) -> tuple[float, float]` returning `(target_speed, steering)`. (See `resources/4students/README.md` for full specification).

Switching Circuits / Maps

Over 20 circuits are available under `maps/<Name>/` (e.g. `Austin`, `Monaco`, `Spa`, `Silverstone...`):

```
from env_simulation import set_map, get_available_maps, reset, get_obs, apply_action,
simulation_step, get_step_info, close

# List all available tracks
print(get_available_maps())

# Switch active track
set_map("Spa")

# Run simulation on the selected map
reset(num_cars=1)
for _ in range(500):
    obs = get_obs(0)
    apply_action(0, 3.0, 0.0)
    simulation_step()
    info = get_step_info()
    if info["lap_complete"][0]:
        break
close()
```

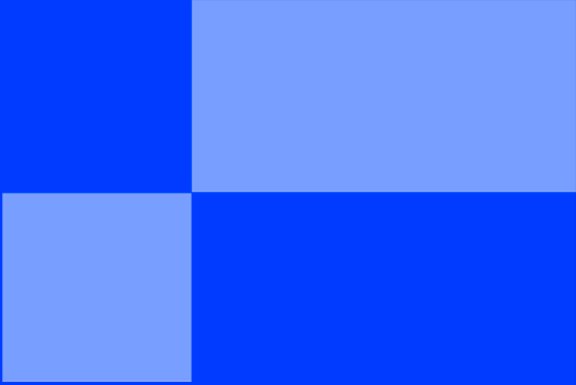
Arguments Reference

`sim_recorder.py`

Argument	Default	Description
<code>--steps N</code>	500	Maximum decision steps
<code>--laps N</code>	2	Target completed laps per car
<code>--num-cars N</code>	1	Number of cars on track (1–4)
<code>--submissions PATH</code>	""	Path to a submission directory or parent folder of team subdirectories
<code>--model PATH</code>	""	Direct path to an ONNX model file for car 0
<code>--controller {auto,submission,random,onnx,pure_pursuit}</code>	auto	Policy controller type
<code>--out PATH</code>	auto	Output <code>.npz</code> file path

`viz_replay.py`

Argument	Default	Description
<code>--npz PATH</code>	""	Path to <code>.npz</code> recording file
<code>--follow</code>	false	Follow car 0 with camera
<code>--speed-factor FLOAT</code>	1.0	Replay speed factor multiplier
<code>--no-display</code>	false	Headless mode (saves snapshot PNG)



{EPITECH}